

# Component & Architecture

## A) Quick Map

|                        |                                                       |
|------------------------|-------------------------------------------------------|
| src/                   |                                                       |
| ├ main.jsx             | # React entry - mounts <Scene />                      |
| ├ Scene.jsx            | # Top-level orchestrator: 3D Canvas + UI wiring       |
| ├ Assets3D/            |                                                       |
| │ └ Book.jsx           | # Page-turning 3D book; raycasting; animations        |
| │ └ TextPage.jsx       | # Builds skinned page mesh with front/back textures   |
| ├ Assets2D/            |                                                       |
| │ └ UI.jsx             | # Pagination, show text, show image; exports pageAtom |
| │ └ ModalImage.jsx     | # MUI modal for images                                |
| │ └ TextOverlay.jsx    | # Readable left/right text pane + TTS                 |
| ├ Context/             |                                                       |
| │ └ GlobalContext.jsx  | # Placeholder global provider (unused at runtime)     |
| ├ InfoData/            |                                                       |
| │ └ PagesContent.jsx   | # usePages() hook; builds page content from API       |
| ├ services/            |                                                       |
| │ └ cavafyService.jsx  | # getAllData(), getImageUrl(); env-driven             |
| └ index.css, fonts.css | # Global styles & font                                |

Key libs: @react-three/fiber, @react-three/drei, three, @mui/material, axios, jotai, maath.

## B) Runtime Flow - What happens in order

1. **main.jsx** mounts <Scene />.
2. **Scene.jsx**:
  - Detects **mobile** via MUI breakpoints; adjusts **camera FOV** (85 on mobile, 45 desktop).
  - Owns transient UI state: `modalImageSrc`, `modalOpen`, `textOverlayOpen`, `currentLeftContent`, `currentRightContent`.
  - Sets up <Canvas> with **lights**, optional <CameraControls> (desktop only), and a floating group.
  - Renders <Book> (3D) and <UI> (2D overlay). Book reports current page's left/right content back via `onCurrentPageChange(left, right)`. UI triggers `onShowTextOverlay()`, and can request `handleCameraReset()`.
  - Renders <ModalImage> and <TextOverlay> with the current content.
3. **Book.jsx**:
  - Pulls data via `usePages()` and the global **pageAtom** (Jotai).
  - Builds a page stack (front/back) using `createTextPageMesh(front, back)` from `TextPage.jsx`.
  - Animates page-turning (easing via **maath**) and manages a **delayed page** index for smooth transitions.
  - Uses **Raycaster** and clickable areas returned from `TextPage` for hit testing; opens image modal through callbacks.
4. **TextPage.jsx**:
  - Converts React **front/back content elements** into **CanvasTextures** (text + images), composes them with a **papyrus** paper texture, and returns a **SkinnedMesh** + **Skeleton** rig (multi-bone) for realistic page curling.
  - Returns **clickable areas** for both sides for later raycasting in `Book`.
5. **UI.jsx**:
  - Exports `pageAtom`; renders pagination and quick actions (text overlay, image modal) based on current page and content presence.
6. **PagesContent.jsx**:
  - Defines a **coverPage** and a `usePages()` hook that fetches & maps backend entries into `pages[]` (shape: { front: ReactNode, back: ReactNode, ...attrs }).
7. **cavafyService.jsx**:
  - `getAllData()` calls `VITE_API_URL + /cavafy-timelines?...` (Strapi-style query). `getImageUrl()` composes `VITE_SERVER` + path with size fallback.